

Technical design: local web chat for Claude Code

Oct 5, 2026 · @John Doe

This is a build spec for a chat page on 127.0.0.1 that talks to your own running Claude Code session. It keeps the chat loop of ECC's Plan Canvas and drops everything about reviewing plans or pull requests. An agent can build it offline: Node.js 18 or later, built-in modules only, no network calls.

What it is and is not

It is:

- A chat window in your browser, served from your own machine, for one person.
- A relay. The page sends text to a small local server; your Claude Code session picks it up and answers.
- Part of your harness: a command-line tool, a skill that teaches Claude Code the loop, and one hook.

It is not:

- A review tool. Nothing is rendered for review, annotated, approved or rejected. No plans, diffs or pull requests.
- A deployed site. The server listens on 127.0.0.1 only and has no accounts.
- A Claude client. It never calls the Claude API and never reads, stores or asks for a token.

There is no sign-in to build. Claude Code stays signed in the way it already is on this machine, and the chat only passes text to it. [Anthropic's terms](#) leave a person free to sign in to unmodified Claude Code with their own subscription.

Changes from ECC's Plan Canvas

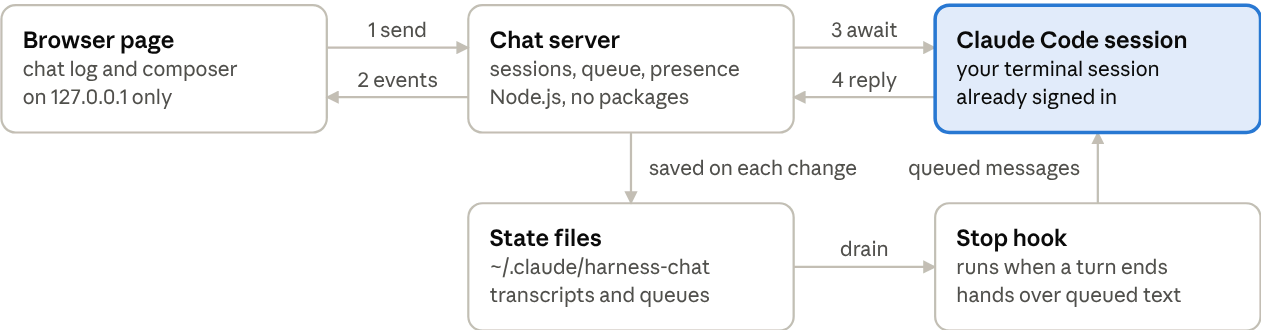
Area	ECC	This design	Reason
Session key	Hash of the reviewed file's path	Hash of the project directory, plus an optional name	No file is under review

Area	ECC	This design	Reason
Review features	Artifact pane, annotations, verdicts, live reload	Removed	Chat only
Await call	GET	POST	A page on another site cannot trigger it unnoticed
Reply text	Command-line argument, cut at 4,000 characters	Standard input, up to 32,000 characters	Replies are long and hold quotes and code
Await result	feedback	messages	Plain naming

How it works

The page never talks to Claude; your Claude Code session does

One chat turn runs steps 1 to 4. The Stop hook is the backstop.



parts and one chat turn · 5 parts

The browser and Claude Code never connect directly. Both talk to the chat server, and only Claude Code holds a login.

One turn, in text for readers who cannot see the picture:

1. **Send.** The page posts the message. The server adds it to the transcript and to the queue, and saves both.
2. **Events.** The server pushes the transcript and the presence state to every open tab.

3. **Await.** The `await` command that Claude Code left running returns the queued messages as JSON and exits. Presence becomes thinking.
4. **Reply.** Claude Code does the work and pipes its answer to the `reply` command. The server adds it to the transcript and pushes events again.
5. Claude Code starts `await` again. Presence becomes listening.

Backstop: if a turn ends with messages queued and no `await` running, the Stop hook drains the queue and hands the messages to Claude Code as its next input.

Files to create

Nine files, all plain Node.js. The names are placeholders; keep the split. The command is called `harness-chat` below and runs `node scripts/chat.js`.

Path	Job
<code>scripts/chat.js</code>	Command-line tool and server entry point. Parses commands, calls the server over loopback, prints JSON
<code>scripts/lib/chat/server.js</code>	HTTP server: pages, event stream, queue, presence, idle shutdown
<code>scripts/lib/chat/sessions.js</code>	Session store on disk with atomic writes
<code>scripts/lib/chat/ui.js</code>	Returns the page HTML, the stylesheet and the browser script as strings
<code>scripts/lib/chat/guard.js</code>	Host and Origin checks used on every request
<code>scripts/lib/chat/launch.js</code>	Opens the default browser on macOS, Windows and Linux
<code>scripts/hooks/chat-pending.js</code>	Stop hook that hands queued messages to Claude Code
<code>skills/chat/SKILL.md</code>	Teaches Claude Code the open, await, reply loop
<code>tests/chat/*.test.js</code>	Store, guard, server and end-to-end tests

Runtime rules:

- Node.js 18 or later, CommonJS, built-in modules only: `http` , `fs` , `path` , `os` , `crypto` , `child_process` , `events` .
- No package install, no CDN, no web fonts, no outbound request of any kind.
- Export a factory from each library file, for example `createChatServer(options)` and `createSessionStore(options)` , so tests can pass a temporary state directory and short timers.

State on disk

Everything lives in `~/.claude/harness-chat/` .

File	Content
<code>sessions.json</code>	Every session with its transcript and queue
<code>server.json</code>	<code>pid</code> , <code>port</code> , <code>version</code> and <code>startedAt</code> of the running server. Written after the server binds, removed on exit
<code>server.log</code>	Output of the detached server

Environment variable	Default	Meaning
<code>HARNESS_CHAT_STATE_DIR</code>	<code>~/.claude/harness-chat</code>	State directory
<code>HARNESS_CHAT_PORT</code>	<code>4611</code>	Loopback port
<code>HARNESS_CHAT_IDLE_MS</code>	<code>18000000</code>	Idle time before the server exits. <code>0</code> or <code>off</code> disables it

Session store

One JSON file holds every session, so a queued message survives a server restart.

```

{
  "counter": 7,
  "sessions": {
    "3fa9c1d2e4b7": {
      "key": "3fa9c1d2e4b7",
      "dir": "/home/me/project",
      "name": "",
      "status": "open",
      "endedBy": null,
      "chat": [
        { "role": "user", "text": "Run the tests", "at": "2026-10-05T09:12:44.120Z" },
        { "role": "agent", "text": "All 42 pass.", "at": "2026-10-05T09:13:10.501Z" }
      ],
      "pending": [],
      "createdAt": "2026-10-05T09:12:40.000Z",
      "updatedAt": "2026-10-05T09:13:10.501Z"
    }
  }
}

```

Rules:

- **Key.** Take the real path of the project directory, with symlinks resolved. Hash it with SHA-256 and keep the first 12 hex characters. When a session name is given, hash the path, a NUL byte and the name.
- **Status.** `open` or `ended`. `endedBy` is `user`, `agent` or `null`.
- **Queue.** `pending` holds messages the agent has not taken: `{ id, text, at }`, with `id` like `m-7` built from `counter`.
- **Transcript.** `chat` holds everything shown in the page, in order. `role` is `user` or `agent`.
- **Writes.** After every change, write the whole state to `sessions.json.tmp`, then rename it over `sessions.json`.
- **Bad file.** A missing or unreadable file gives an empty store. It must never stop the server.

Operation	Behaviour
<code>open(dir, name, reopen)</code>	Create or resume. If the user ended the session and <code>reopen</code> is false, return <code>refused</code> and change nothing. Otherwise set <code>open</code> and clear <code>endedBy</code>
<code>addUserMessage(key, text, endSession)</code>	Return null if the session is ended. Append to <code>chat</code> and to <code>pending</code> . With <code>endSession</code> , also mark the session ended by the user
<code>takeMessages(key)</code>	Unknown key: <code>missing</code> . Queue has items: return them all as <code>messages</code> and empty the queue; add <code>sessionEnded</code> and <code>endedBy</code> if the session is ended. Empty queue and ended: <code>ended</code> . Otherwise <code>waiting</code>
<code>addAgentReply(key, text)</code>	Append an <code>agent</code> entry to <code>chat</code>
<code>end(key, by)</code>	Set <code>ended</code> and <code>endedBy</code>
<code>list()</code>	For each session: key, dir, name, status, endedBy, pending count, updatedAt

`takeMessages` gives a batch to exactly one caller. A second caller a moment later gets `waiting`.

HTTP interface

The server speaks plain HTTP on 127.0.0.1. The page and the command-line tool are its only clients.

Rules for every request

1. Listen on `127.0.0.1` only.
2. Answer 403 unless the Host header names `127.0.0.1`, `localhost` or `:::1`, with any port.
3. Answer 403 if an Origin header is present and its host is not one of those three. A missing Origin is fine: the command-line tool and plain page loads send none.
4. Every call that changes anything is a POST. A page on another site can only send a POST with its own Origin, which rule 3 rejects.

- 5. Read JSON bodies up to 1 MiB. A larger or invalid body gets 400.
- 6. A session key must match `^[a-f0-9]{12}$`. An unknown key gets 404.
- 7. Message and reply text must be a non-empty string of at most 32,000 characters, else 400.
- 8. Send `cache-control: no-store` on every response. Errors are JSON: `{ "error": "..."}.`
- 9. HTML pages also send `content-security-policy: default-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:.`

The guard for rules 2 and 3:

```
const LOOPBACK = new Set(['127.0.0.1', 'localhost', ':::1']);

// "127.0.0.1:4611" -> "127.0.0.1", ":::1:4611" -> ":::1"
function hostOf(header) {
  const m = /^(\[^\]]+\)|\[^\]]+(?:::\d{1,5})?$/ .exec(String(header || ''))
    .trim());
  return m ? m[1].toLowerCase() : null;
}

function allowed(req) {
  if (!LOOPBACK.has(hostOf(req.headers.host))) return false;
  const origin = req.headers.origin;
  if (!origin) return true;
  try { return LOOPBACK.has(new URL(origin).hostname.toLowerCase()); }
  catch { return false; }
}
```

Endpoints

Method and path	Caller	Request body	Response
GET /health	Tool	None	{ ok: true, app: "harness-chat", version }
POST /shutdown	Tool	None	{ status: "stopping" }, then the server exits

Method and path	Caller	Request body	Response
GET /	Browser	None	HTML list of sessions with links
GET /chat/<key>	Browser	None	HTML chat page
GET /chat.css and GET /client.js	Browser	None	Stylesheet and browser script
GET /events/<key>	Browser	None	Event stream, described below
POST /api/session/<key>/messages	Browser	{ text, endSession? }	{ status: "queued", id, pending, presence } .409 if the session is ended
POST /api/session/<key>/end	Browser	None	{ status: "ended", endedBy: "user" }
POST /api/sessions	Tool	{ dir, name?, reopen? }	{ status: "open", key, url } .409 with { status: "user-ended", key } when refused
GET /api/sessions	Tool	None	{ sessions: [...] } from list()
POST /api/await	Tool	{ key, timeoutMs? }	Long poll, described below
POST /api/session/<key>/reply	Tool	{ text }	{ status: "sent", at }
POST /api/session/<key>/typing	Tool	{ state }, one of thinking, typing, idle	{ status: "ok", presence }
POST /api/session/<key>/agent-end	Tool	None	{ status: "ended", endedBy: "agent" }

After a message, a reply, a typing call or an end, the server broadcasts to the session's event streams.

Event stream

```
HTTP/1.1 200 OK
content-type: text/event-stream
cache-control: no-store
connection: keep-alive

event: chat-sync
data: {"chat":[{"role":"user","text":"Run the tests","at":"..."}]}

event: presence
data: {"state":"thinking"}

: ping
```

- On connect, send `chat-sync` and then `presence`.
- `chat-sync` always carries the whole transcript. Send it whenever the transcript changes.
- `presence` carries `{ state }`. Send it whenever the state may have changed.
- `ended` carries `{ endedBy }`. Send it once when the session ends.
- Send the comment line `: ping` every 25 s to keep the connection open.
- Keep a set of open responses per session. Remove a response when its request closes.

Long poll

`POST /api/await` is how the agent waits for the user.

1. Call `takeMessages`. If the result is not `waiting`, answer at once with that JSON.
2. Otherwise send status 200 with `content-type: application/json`, write one space, and keep the response open.
3. Add 1 to the session's await count and broadcast presence.
4. Write one more space every 15 s as a heartbeat.
5. When a message arrives or the session ends, call `takeMessages` again. If the result is not `waiting`, write it and end the response.
6. If `timeoutMs` passes first, write `{"status":"waiting"}` and end. `timeoutMs` of 0 means answer at once.

7. If the client disconnects, clean up and write nothing.
8. On shutdown, answer every parked call with `{"status": "waiting", "note": "server stopping, run await again"}`.
9. Whenever a parked call ends, subtract 1 from the await count and broadcast presence.

The caller trims the leading spaces before parsing. Use one `EventEmitter` with an event per key to wake parked calls.

```
{ "status": "messages", "items": [ { "id": "m-7", "text": "Run the tests",
  "at": "..." } ] }
{ "status": "messages", "items": [ { "id": "m-8", "text": "Thanks, bye",
  "at": "..." } ], "sessionEnded": true, "endedBy": "user" }
{ "status": "ended", "endedBy": "user" }
{ "status": "waiting" }
{ "status": "missing" }
```

Presence

Presence tells the page what Claude Code is doing, and never claims more than the server knows.

State	Meaning	Label in the page
ended	The session is closed	session ended
typing	The agent signalled that a reply is seconds away	Claude is typing...
thinking	The agent took messages and is working	Claude is thinking...
listening	An await call is parked right now	Claude is listening
queued	Messages are waiting and nobody is listening	queued for Claude
waiting	Nothing queued, nobody listening	Claude is not connected

The server keeps three small maps in memory, keyed by session: `workingAt`, `typingAt` and `awaitCount`. It computes the state on demand, and the first true line wins:

```
function presenceFor(key, now = Date.now()) {
  const s = store.get(key);
  if (!s || s.status === 'ended') return 'ended';
  if (typingAt.has(key) && now - typingAt.get(key) < 30000) return 'typing';
  if (workingAt.has(key) && now - workingAt.get(key) < 90000) return
  'thinking';
  if ((awaitCount.get(key) || 0) > 0) return 'listening';
  return s.pending.length > 0 ? 'queued' : 'waiting';
}
```

Event	Effect
An await call hands over a batch	Set <code>workingAt</code> to now, clear <code>typingAt</code>
An await call parks on an empty queue	Clear both, add 1 to <code>awaitCount</code>
A parked await call ends	Subtract 1 from <code>awaitCount</code>
Typing call with <code>thinking</code>	Set <code>workingAt</code> to now, clear <code>typingAt</code>
Typing call with <code>typing</code>	Set <code>typingAt</code> to now
Typing call with <code>idle</code> , or a reply	Clear both
The session ends	Clear both

Broadcast presence after each event. Every 5 s, recompute the state for each session that has an open event stream and broadcast only if it changed. That sweep is what clears a thinking or typing state after it expires, so a crashed agent decays to `queued` or `waiting` by itself.

Make the three timings options of the server factory: 90 s for thinking, 30 s for typing, 5 s for the sweep.

Command-line tool

`harness-chat` is the only thing Claude Code runs. It acts on the session of the current directory, so no command takes an ID.

Command	What it does	Prints
<code>harness-chat</code>	Shows the server address, version and sessions	<code>{ server, version, sessions }</code>
<code>harness-chat open [--name n] [--no-open] [--reopen]</code>	Starts the server if needed, opens or resumes the session, launches the browser	<code>{ status: "open", url, key }</code> , or <code>{ status: "user-ended" }</code> when refused
<code>harness-chat await [--name n] [--timeout-ms n]</code>	Blocks until the user writes or the session ends	An await result
<code>harness-chat reply [--name n] [--text t]</code>	Sends a reply. Reads standard input when <code>--text</code> is absent	<code>{ status: "sent", at }</code>
<code>harness-chat typing [--name n] [--state s]</code>	Sets <code>thinking</code> , <code>typing</code> or <code>idle</code> . Default <code>typing</code>	<code>{ status: "ok", presence }</code>
<code>harness-chat pending</code>	Lists sessions with undelivered messages, read straight from the state file	<code>{ status, sessions }</code> , with status <code>pending</code> or <code>clear</code>
<code>harness-chat end [--name n]</code>	Ends the session as the agent	<code>{ status: "ended", endedBy: "agent" }</code>
<code>harness-chat stop</code>	Shuts the server down	<code>{ status: "stopping" }</code>
<code>harness-chat server [--port n]</code>	Runs the server in the foreground	Log lines on standard error

Output rules:

- Results go to standard output as JSON. Progress notes go to standard error, so standard output stays parseable.
- An error prints `{ "error": "...", }` and exits with code 1.
- With no server running, `await`, `reply`, `typing` and `end` print `{ "status": "no-server" }`.
- `await` adds a `next_step` string that tells the agent what to do, so the loop holds even when the skill is not loaded.

Await result	next_step text
<code>messages</code>	Answer in the chat with <code>harness-chat reply</code> , then run <code>harness-chat await</code> again in the background.
<code>messages</code> with <code>sessionEnded</code>	The user sent this and closed the chat. Do what it asks, report in the terminal, and do not reopen the chat.
<code>ended</code> by the user	The user closed the chat. Stop listening and do not reopen it unless asked.
<code>waiting</code>	No message yet. Run <code>harness-chat await</code> again.

Server lifecycle

1. `open` reads the port from `server.json` , or uses the default, and calls `GET /health` .
2. Healthy with the same version: reuse the server.
3. Healthy with another version: call `POST /shutdown` , wait until health fails, then start a new server.
4. Not healthy: start `node scripts/chat.js server --port <port>` as a detached child. Append its output to `server.log` and call `unref()` .
5. Poll `/health` every 100 ms for up to 5 s. On failure, exit with an error that names `server.log` .
6. The server loads `sessions.json` at start, so transcripts and queued messages come back.
7. The server exits by itself after 30 minutes with no open event stream and no parked await call.
8. On SIGINT, SIGTERM or `/shutdown` , the server answers parked await calls, closes event streams, removes `server.json` and exits.

`version` comes from the harness's `package.json` . The check in step 3 keeps the page, the server and the tool on one protocol after an update.

Opening the browser

The URL is always `http://127.0.0.1:<port>/chat/<key>` . Check that shape before launching, and pass the URL as one argument, never inside a shell string.

Platform	Command
macOS	<code>open <url></code>
Linux	<code>xdg-open <url></code>
Windows	<code>cmd /c start "" <url></code>

Spawn it detached, handle the child's `error` event, and never fail `open` because the browser did not start. The printed URL is the fallback.

Browser page

One page, no framework and no build step. `ui.js` returns three strings: the HTML shell, the stylesheet and the browser script.

Part	Content
Header bar	Title, project folder name, presence pill, theme button, End chat button
Message log	User messages on the right, Claude's on the left, each with sender and local time
Composer	Text area, Send button, one-line status

Boot:

- The server writes `{ key, dir, status, endedBy, chat }` as JSON into a `<script type="application/json" id="boot">` tag. Replace every `<` in that JSON with `\u003c`.
- The script reads it, draws the log, then opens `new EventSource('/events/' + key)`.

Event	Page reaction
<code>chat-sync</code>	Redraw the whole log from the transcript
<code>presence</code>	Update the pill and the activity bubble
<code>ended</code>	Disable the composer, show who ended the chat, close the stream

Event	Page reaction
Stream error	Show "chat server offline" in the pill. The browser reconnects by itself

Sending:

1. Ignore Send while a send is in flight or the session is ended.
2. POST `{ text }` to `/api/session/<key>/messages`.
3. On success, clear the text area and apply the returned presence.
4. Set the status line to "Delivered. Claude has it." when that presence is `thinking` or `typing`. Otherwise set it to "Queued. Claude picks this up when it next listens."
5. On failure, keep the text and show "Send failed. Is the chat server running?"

Rules:

- Enter sends. Shift+Enter adds a line.
- Put message text into the page with `textContent` and style it with `white-space: pre-wrap`. Never assign message text to `innerHTML`.
- Keep the log scrolled to the bottom only when the reader is already within 40 px of the bottom.
- Show a three-dot bubble while the state is `thinking` or `typing`. Show a plain note while it is `queued`: "Your message is in the queue. Claude Code is not listening right now."
- Give the bubble and the note `role="status"`, and stop the dot animation under `prefers-reduced-motion`.
- Dark theme by default, light on toggle. Save the choice in `localStorage` inside `try` and `catch`, because storage access can throw on loopback pages.
- End chat asks for confirmation, then posts to `/api/session/<key>/end`.
- An empty log reads: "Type a message. It goes to your Claude Code session."

Optional code blocks: Claude's replies often contain fenced code. To show it well without parsing HTML, split the text on lines that start with three backticks. Create a `pre` element for each code part and a `div` for each text part, and fill both with `textContent`.

Teaching Claude Code the loop

Three pieces make the chat work inside a session: a skill, a permission rule and a Stop hook.

Skill

Save this as `skills/chat/SKILL.md`:

```
---
name: chat
description: Talk with the user through the local chat page. Use when the
user asks to open the chat or to chat in the browser, or when chat messages
are waiting.
---
```

Local chat

The user types in a browser page on this machine. You receive their messages with `harness-chat await` and answer with `harness-chat reply`.

1. Open the chat: `harness-chat open`. It prints the page URL.
2. Listen: run `harness-chat await` as a background task and leave it running.
It exits when the user writes, and prints JSON.
3. Read `items`. Treat each `text` as a message from the user, exactly as if they had typed it in the terminal.
4. During work that takes more than a minute, run
`harness-chat typing --state thinking` about once a minute.
5. Answer in the chat, every time, even with one line:

```
harness-chat reply <<'EOF'
your answer
EOF
```
6. Start `harness-chat await` again at once.

Stop listening when await returns "status": "ended" or "sessionEnded": true. Do not reopen a chat the user ended unless they ask; then run
`harness-chat open --reopen`.

Never end your turn while the chat is open and no await is running. The user would see a sent message and no answer.

If you are unsure whether you missed something, run ``harness-chat pending``.

- In Claude Code, a background task is a Bash call with `run_in_background: true`. It keeps the loop alive across turns. A foreground await also works, but only until the tool's time limit.
- The quoted heredoc marker keeps quotes, backticks and dollar signs in the reply intact.

Install and permission rule

`harness-chat` must resolve from any directory, because the current directory is the project under discussion. Add a `bin` entry for `scripts/chat.js` and run `npm link` in the harness, or call the script by absolute path.

Without a permission rule, Claude Code asks in the terminal before every command. Add rules of this shape to `.claude/settings.json`, then confirm them with `/permissions`:

```
{
  "permissions": {
    "allow": ["Bash(harness-chat)", "Bash(harness-chat *)"]
  }
}
```

Stop hook

The hook covers one gap: a turn ends with messages queued and no await running.

Register it in `.claude/settings.json`; the timeout is in seconds:

```
{
  "hooks": {
    "Stop": [
      {
        "matcher": ".*",
        "hooks": [
          { "type": "command", "command": "node
/absolute/path/to/scripts/hooks/chat-pending.js", "timeout": 30 }
        ]
      }
    ]
  }
}
```

What `chat-pending.js` does:

1. Read all of standard input and parse it as JSON. It carries `cwd` and `stop_hook_active`.
2. If parsing fails or `stop_hook_active` is true, exit 0 with no output. Blocking twice in a row can wedge a session.
3. Read `sessions.json`. Pick sessions that are not ended, have pending messages, and whose `dir` is the real path of `cwd`.
4. Drain each one through the server: `POST /api/await` with `{ key, timeoutMs: 0 }` and a 1s request timeout. The server owns the file while it runs.
5. If `server.json` is absent or the connection is refused, no server is running. Then empty `pending` in the file directly, with the same tmp-and-rename write.
6. If nothing was drained, exit 0 with no output.
7. Otherwise print one JSON object to standard output and exit 0.
8. On any error, exit 0 with no output. A hook must never break the session.

```
{
  "decision": "block",
  "reason": "Local chat: the user sent messages that never reached you.\n-
Run the tests again\nAnswer with `harness-chat reply`, then run `harness-chat
await` in the background."
}
```

`decision: block` stops the turn from ending and hands `reason` to Claude Code as its next input. List at most 20 messages in it.

ECC also registers a `SessionStart` hook that tells a fresh session about open chats. Add that only after the core works.

Tests and a manual check

Run everything with `node --test tests/chat/`. No test may touch the network or the real state directory.

Area	Cases
Store	The key is the same for every spelling of a path. A user-ended session refuses a plain reopen and accepts <code>reopen</code> . An agent-ended session reopens. <code>takeMessages</code> drains once, then returns <code>waiting</code> . An ended session refuses new messages. A final batch carries <code>sessionEnded</code> . Pending messages survive a reload from disk. A corrupt file gives an empty store
Guard	A foreign Host gets 403. A foreign Origin gets 403. A missing Origin passes. <code>[::1]</code> with a port passes
Server	A parked await returns when a message is posted. Of two parked awaits, one gets the batch. Thinking and typing expire with short test timers. A new event stream gets <code>chat-sync</code> then <code>presence</code> . Shutdown answers parked awaits with <code>waiting</code> . A body over 1 MiB gets 400. Text over 32,000 characters gets 400
Tool	<code>open --no-open</code> , then <code>await --timeout-ms 200</code> prints <code>waiting</code> . After a posted message, <code>await</code> prints <code>messages</code> . <code>reply</code> from standard input adds an agent entry to the transcript
Hook	With a queued message and no server, it prints a <code>block</code> decision and empties the queue. With <code>stop_hook_active</code> true, it prints nothing

Manual check, in a scratch directory:

```
harness-chat open --no-open          # prints url and key
curl -s http://127.0.0.1:4611/health # {"ok":true,...}
harness-chat await --timeout-ms 500  # {"status":"waiting",...}
curl -s -X POST http://127.0.0.1:4611/api/session/KEY/messages \
  -H 'content-type: application/json' -d '{"text":"hello"}'
harness-chat await                   # {"status":"messages",...}
echo "hi from the agent" | harness-chat reply
curl -s -H 'host: evil.example' http://127.0.0.1:4611/health # 403
harness-chat stop
```

Done when:

- ☐ All tests pass with the network unplugged.
- ☐ A message typed in the page reaches a parked await within a second.
- ☐ A reply piped to `harness-chat reply` shows in the page without a reload.
- ☐ A message sent with no await parked shows as queued, survives `stop` and `open`, and arrives exactly once.
- ☐ After End chat, `open` prints `user-ended` and `await` prints `ended`.
- ☐ The code contains no read of a token, an API key or a credential file.

Limits

- Permission prompts stay in the terminal. If Claude Code asks before running a tool, the page shows thinking until you answer there.
- No streaming. A reply appears whole when Claude Code sends it.
- A message is picked up only while an await is parked, or at the end of a turn through the Stop hook. With Claude Code closed, messages wait in the queue.
- One Claude Code session per chat. Two sessions awaiting the same chat split the messages between them.
- Any program running as you on this machine can call the server. That is the same trust level as your terminal.

Ready-made options

Two Anthropic features cover similar ground as of 5 Oct 2026. They need no build, but each has a catch.

Option	What it is	Catch
Fakechat channel	An Anthropic demo plugin: a chat page on localhost that pushes messages into a running Claude Code session	Channels are a research preview and need Bun and the <code>--channels</code> flag
Remote Control	Drives your local session from <code>claude.ai/code</code> or the Claude mobile app	Needs a subscription login, and the transcript is stored on Anthropic's servers while connected

Sources

For the human reader. The build needs none of these links.

- [ECC repository](#) at commit `ef648e0`, release 2.2.3: Plan Canvas server, session store, page, command-line tool, skill, Stop hook and hook registration.
- Claude Code docs: [Legal and compliance](#), [Channels](#), [Remote Control](#).